

Limits of Differential Testing: Three Rigid-Body Dynamics Engines Under Library-Independent Adjudication

Noah Parsons^a

^aIndependent Researcher, Newcastle, Wyoming, United States

Abstract

Differential testing detects the failures that independent implementations make independently; it is structurally blind to the failures they share. Such failures exist in widely used scientific software and are not reachable by adding implementations. Three rigid-body dynamics engines — a symbolic compiler, a computer-algebra package, and a recursive articulated-body library — were adjudicated against a closed-form reference implemented in NumPy alone, sharing no derivation code with any of them. A 55-case adversarial suite was frozen before measurement and extended to three structurally distinct mechanisms; integration was pinned to one scheme for every engine. The engines agree with the reference and with one another to within 3×10^{-14} on the equations of motion, and no engine returned a wrong answer while reporting success on any adjudicated case. An apparent divergence in reach measured how each engine had been driven rather than what it could do: given equal budgets, all three answer 80-coordinate systems. One failure was observed, and every implementation shares it: at an exact unstable

Email address: `parsons.m.noah@gmail.com` (Noah Parsons)

URL: <https://orcid.org/0009-0000-7224-6040> (Noah Parsons)

equilibrium every engine, and the reference, reports success while returning approximately 20 radians of motion for a configuration whose exact solution does not move, and none warns. Divergence time grows linearly with working precision and survives to 150 decimal digits, so it is structural: numerically induced rather than a defect in any implementation. The contribution is methodological: a concrete failure that no amount of implementation diversity would expose, and a reusable design for distinguishing shared failures from independent ones.

Keywords: differential testing, test oracle, shared failure, N-version programming, scientific software, code verification, floating-point arithmetic

1. Introduction

1.1. The problem

Scientific results increasingly rest on software that derives the equations governing a physical system and integrates them forward. A researcher modelling a joint, a mechanism, or a robot selects an engine, states the system, and takes the resulting trajectory as an account of what the physical system would do. The correctness of that account is rarely tested directly. It is inferred from the engine’s reputation, from its test suite, and from the absence of an error message.

Where that inference is tested, it is usually tested by comparison: run two implementations and treat disagreement as evidence of a defect. The method is powerful and its limit is well known in principle — it can only find failures the implementations do not share. What that limit means in practice is

14 less often examined, because it requires finding a shared failure and showing
15 that no amount of implementation diversity would have exposed it. This
16 paper does that, and the result runs against the premise: the shared failure
17 reported here originates in floating-point arithmetic rather than in anyone's
18 code, so every implementation exhibits it, including a reference written from
19 first principles specifically to be independent.

20 1.2. *The failure mode that motivates the study*

21 An engine that raises an exception is not dangerous; the researcher knows
22 to stop. The dangerous case is the engine that returns a plausible trajectory
23 that is wrong, with no indication that anything has gone amiss. Call this a
24 *silent failure*. A silent failure is by construction the case nobody thought to
25 check for, which is why it survives in test suites written by the same people
26 who wrote the implementation.

27 1.3. *Two claims, often conflated*

28 Discussion of this failure mode tends to run together two distinct propo-
29 sitions:

- 30 (A) These engines can report success while returning wrong physics.
- 31 (B) Engines *differ* in whether they do so, such that comparing implementa-
32 tions reveals incorrectness.

33 Claim (B) is the premise of differential testing: if two independent implemen-
34 tations disagree, at least one is wrong, and the disagreement is the signal.
35 This paper set out to test (B) and did not find it.

36 *1.4. Research questions*

37 Three questions follow from the distinction, each answered by a distinct
38 part of the study:

39 **RQ1** Under adjudication by a reference sharing no derivation code with
40 any engine, do independently developed rigid-body dynamics engines
41 disagree on correctness? (§5)

42 **RQ2** Does a comparison of such engines measure the engines, or the way
43 they were driven? (§6)

44 **RQ3** Are there failures every implementation exhibits identically — includ-
45 ing a reference constructed to be independent — and what follows
46 for differential testing if there are? (§7, §10)

47 RQ1 is answered in the negative, RQ2 in the second sense, and RQ3 by
48 exhibiting one such failure.

49 *1.5. What was done*

50 Three independently developed engines were compared against a closed-
51 form reference implemented in NumPy alone, sharing no derivation code with
52 any of them. A 55-case adversarial matrix was frozen before measurement
53 under a governance rule forbidding removal of cases after their results were
54 seen. Three mechanism families were examined, chosen so that each degen-
55 erates by a different mechanism. Integration was pinned to one scheme with
56 identical tolerances for every engine. The author’s conflict of interest is dis-
57 closed in §3.7, and again in the Disclosure statement following the conclusion,
58 and is mitigated structurally rather than by declaration.

59 *1.6. What was found*

60 Agreement to the limit of double-precision arithmetic on every adjudi-
61 cated case in every family, and no correctness disagreement on any axis mea-
62 sured. An apparent divergence in *reach* — the largest system each engine
63 would answer — which proved on examination to measure how each engine
64 had been driven rather than what it could do: under equal budgets all three
65 answer 80-coordinate systems (§6.1). One failure, at an unstable equilibrium,
66 shared by all three engines and by the independent reference alike.

67 *1.7. Why a negative result here is informative*

68 The measurement was designed to detect disagreement, and the evidence
69 that it can is its record against itself. It detected *four* faults in its own
70 construction — three convention mismatches and one mistaken adjudication
71 criterion — before any of them reached the reported results (§3.6, §9). Each
72 was caught by consulting an independent artefact rather than by reasoning
73 further, and each would otherwise have produced a fabricated finding against
74 widely used software. An instrument that catches four faults in itself is not
75 one that would miss a correctness disagreement of the size differential testing
76 exists to expose.

77 A second argument was originally made here and is withdrawn. The
78 instrument appeared to resolve a large capability difference between the en-
79 gines, and that was offered as further evidence of its sensitivity. The differ-
80 ence was an artefact of how each engine was driven (§6). It is recorded here
81 because a study of measurements that mislead should not quietly discard
82 its own: the reach comparison was confounded, the confound survived the
83 freeze, and it was found only when each engine was driven along more than

84 one path. The correctness measurements are unaffected, because they adju-
85 dicate each engine against an independent reference at fixed size rather than
86 against each other.

87 1.8. Contributions

- 88 1. A differential comparison of three independently developed rigid-body
89 dynamics engines under adjudication by a reference sharing no deriva-
90 tion code with any of them, across three mechanism families and both
91 dynamical regimes.
- 92 2. A negative result: no correctness disagreement was found on any axis
93 measured. An apparent separation by reach proved to be an artefact of
94 driving style; under equal budgets the engines separate by speed alone.
- 95 3. A shared blind spot at unstable equilibria, present in all three engines *and*
96 in the independent reference, showing that the failure mode is a property
97 of finite-precision dynamics rather than of any implementation.
- 98 4. A methodological account — frozen case matrix, library-independent
99 referee, integrator pinning, maintainer disclosure — reusable for cross-
100 implementation comparison in other computational domains.
- 101 5. A transferable diagnostic for convention mismatch in cross-implemen-
102 tation comparison, derived from three independent instances encountered
103 in this study, *and its measured limits*: validation against injected faults
104 of known type shows the rule is sufficient but not necessary.

105 2. Related work

106 2.1. *Differential testing and the independence assumption*

107 Differential testing — running two or more implementations of the same
108 specification on the same input and treating any difference in output as evi-
109 dence of a defect — was named and systematised by McKeeman [15], and has
110 since become a standard instrument wherever a specification admits multiple
111 implementations. Its appeal is that it dissolves the test oracle problem: no
112 statement of correct behaviour is needed, only two implementations and a
113 disagreement.

114 The instrument has been productive. Csmith [6] generated random C pro-
115 grams and cross-checked independent compilers against one another, finding
116 over three hundred defects in GCC and LLVM; equivalence modulo inputs [3]
117 extended the idea by deriving variants guaranteed to behave identically on a
118 given input, and found hundreds more. The approach has since been carried
119 to deep learning libraries, where CRADLE [5] cross-validates the same model
120 across Keras backends, and to a range of other systems admitting multiple
121 implementations; Chen et al. [2] survey the compiler-testing line in full.

122 **What these studies share is the setting this paper departs from.**
123 Each operates where no reference answer exists — there is no closed form for
124 what a C program should compile to — so cross-checking implementations is
125 not merely convenient but the only oracle available. Each therefore detects,
126 by construction, precisely the defects that implementations do *not* share.
127 The systems studied here are unusual in admitting a closed-form answer,
128 which makes it possible to ask the complementary question: what does cross-
129 checking miss? The answer requires an oracle those studies could not have

130 had.

131 Its premise is that independent implementations fail independently. That
132 premise has been tested directly, and it failed. Knight and Leveson [14] pre-
133 pared 27 versions of one program independently from a common specification
134 at two universities and subjected them to one million tests. The versions were
135 individually very reliable, but the number of tests on which more than one
136 version failed was substantially greater than independence predicts. Their
137 result is the standard caution against N -version programming, and it is the
138 closest antecedent of the present paper’s central finding.

139 The mechanism differs, and the difference is the contribution. Knight and
140 Leveson attributed correlated failure to shared human misconception: inde-
141 pendent programmers reading the same specification make the same mistakes,
142 because some parts of a problem are harder to think about than others. The
143 correlated failure reported in §7 has no human cause at all. Every implemen-
144 tation examined here — including a reference written from first principles
145 specifically to be independent — fails identically at an unstable equilibrium,
146 because the failure originates in floating-point arithmetic rather than in any-
147 one’s reasoning. Shared misconceptions can in principle be diluted by adding
148 more independent authors. Shared arithmetic cannot.

149 2.2. The oracle problem and scientific software

150 Barr et al. [8] survey the test oracle problem and classify the partial
151 remedies available when no exact oracle exists: derived oracles, metamorphic
152 relations, and *pseudo-oracles* — independently produced implementations
153 used as a standard of comparison. The reference used here is a pseudo-
154 oracle in that sense, distinguished by an explicit and checkable independence

155 criterion: it shares no derivation code with any engine under test, which is
156 a stronger condition than independent authorship and a much stronger one
157 than independent implementation within a common ecosystem.

158 Kanewala and Bieman [13] review testing practice in scientific software
159 specifically and identify the oracle problem as one of its two defining diffi-
160 culties: for software whose purpose is to compute a result nobody knows in
161 advance, the expected output is frequently unavailable by construction. This
162 is precisely the situation of a dynamics engine asked for the trajectory of a
163 mechanism that has never been built.

164 The most direct empirical precedent is Hatton and Roberts [11], who ran
165 nine independently developed seismic data processing packages on identical
166 inputs and found agreement to roughly one significant figure — a disagree-
167 ment so large that it called into question published results in the field. Their
168 study and this one share a design and reach opposite conclusions, which
169 requires explanation rather than assertion. Three differences are plausible.
170 Their packages implemented a long processing pipeline whose specification
171 left considerable latitude, whereas the equations of motion of a rigid mech-
172 anism are mathematically determined once the Lagrangian is fixed. Their
173 comparison was engine-against-engine, with no external standard, so disagree-
174 ment could be localised but not attributed. And three decades separate the
175 two studies. The present result should therefore be read narrowly: rigid-
176 body dynamics is a domain where the specification is unusually tight, and
177 the finding of agreement does not transfer to computational domains where
178 it is not.

179 2.3. *Code verification, and where this study sits in it*

180 Computational science has its own vocabulary for the question asked here,
181 and readers arriving from that tradition will expect the study to be located
182 in it. Oberkampf and Trucano [16], and the monograph of Oberkampf and
183 Roy [17], draw the organising distinction: *code verification* asks whether
184 a program correctly solves the equations it claims to solve, *validation* asks
185 whether those equations describe reality. Roache [18] puts it sharply — verifi-
186 cation is mathematics, validation is physics — and ASME V&V10 [1] codifies
187 the process.

188 **In that taxonomy this is a code-verification study and nothing**
189 **else.** No measurement is compared against an experiment, so no validation
190 claim is made. Three consequences bear on how the results should be read.

191 The standard instrument of code verification is the method of manufac-
192 tured solutions [19], which supplies a known answer for equations possessing
193 no useful closed form. The systems here do not need it: a rigid mechanism
194 of modest size has a genuine closed-form acceleration field, so the referee
195 of §3.2 serves the same purpose without manufacturing anything. Where
196 closed forms are unavailable — contact, friction, large flexible systems —
197 manufactured solutions remain the appropriate tool and this design does not
198 extend.

199 Code verification conventionally rests on *order-of-accuracy* testing, which
200 this study deliberately does not perform: pinning one integrator across all
201 engines (§3.4) is what makes the derivations comparable and is also what
202 forecloses such a claim. What is verified is the right-hand side each engine
203 derives, not the discretisation applied to it.

204 Finally, the frozen case matrix and its governance rule (§3.3) are an in-
 205 formal instance of the pre-specified verification plan ASME V&V10 requires.
 206 The motivation is one software engineering shares: a result is worth little if
 207 the tests it passed were chosen after seeing which ones it failed.

208 2.4. Verification and validation of multibody codes

209 The multibody dynamics community maintains its own verification in-
 210 frastructure. The IFToMM Technical Committee for Multibody Dynamics
 211 hosts a library of computational benchmark problems [12] intended to stan-
 212 dardise comparison of simulation software on both accuracy and efficiency.
 213 That effort is complementary to this one but differs in its notion of ground
 214 truth: benchmark solutions are established by consensus among submitted
 215 solvers, which is a reasonable practice and a different epistemic basis from a
 216 closed-form reference that shares no code with any participant.

217 Nearest in subject matter, González et al. [10] examine the behaviour of
 218 augmented-Lagrangian and Hamiltonian formulations for constrained multi-
 219 body systems in the neighbourhood of singular configurations, identifying
 220 the sources of numerical difficulty there and comparing formulations for ro-
 221 bustness. Their singular configurations are the analogue of the dead centres
 222 of §4.2. The distinction is that they compare *formulations* within a con-
 223 trolled implementation, isolating the effect of mathematical choice, whereas
 224 this study compares *implementations* of nominally the same formulation, iso-
 225 lating the effect of engineering. The two questions are separable and both
 226 worth asking; a formulation that is robust in principle can still be imple-
 227 mented incorrectly, and this paper measures the second.

228 2.5. Comparisons of dynamics engines

229 Erez, Tassa and Todorov [9] compare five simulation engines — among
230 them Bullet, MuJoCo, ODE and PhysX — on quantitative measures chosen
231 for robotics workloads, instantiating the same model in each and running
232 side-by-side. Their principal conclusion is that each engine performs best
233 on the class of system it was designed and optimised for. That finding is
234 reproduced incidentally here, and by a different measurement: the recursive
235 articulated-body library returns the largest residuals of the three engines on
236 long serial chains and the smallest on the two-body tree (§5.6), an ordering
237 that reverses with the shape of the problem rather than with the quality of
238 the engine.

239 The difference in aim is what leaves room for the present study. Erez
240 et al. measure speed, stability, and behaviour under contact — properties
241 that matter for whether an engine is usable — and adjudicate against no
242 external standard, because for the contact-rich systems they consider none
243 exists in closed form. This paper asks a narrower question, on systems chosen
244 precisely because closed forms do exist: not whether the engines are usable
245 but whether their equations are right, judged against a derivation that shares
246 nothing with any of them.

247 2.6. The gap

248 Prior work establishes that differential testing is powerful, that its inde-
249 pendence premise is unreliable in the presence of shared human error, that
250 scientific software has historically disagreed badly under N -version compari-
251 son, that dynamics engines differ measurably in performance, and that code

252 verification has a mature methodology built largely around manufactured so-
253 lutions and order-of-accuracy testing. What has not been reported, to the
254 author’s knowledge, is a comparison of symbolic and numerical dynamics
255 engines adjudicated by a closed-form reference sharing no library with any
256 of them, over a case matrix frozen before measurement, with the resulting
257 disagreements characterised. That is the gap this paper addresses, and the
258 result — agreement everywhere except in a failure the referee shares — is
259 informative chiefly because the design would have detected the alternative.

260 **3. Method**

261 *3.1. The engines under test*

262 Three engines were selected on the criterion that each derives the equa-
263 tions of motion of a mechanical system by a different route, and that no two
264 share a substantial body of code.

265 Versions were pinned for the duration of the study. Pinning is not a
266 formality: an engine that changes between the derivation and integration
267 phases of a comparison invalidates the comparison, and the version under test
268 must be recoverable by a reader attempting to reproduce the measurement.

269 Each engine was exercised through the pathways it exposes. Two support
270 a Lagrangian formulation; one additionally supports a Hamiltonian formula-
271 tion; one constructs models through a model-building interface rather than
272 from a Lagrangian. This asymmetry is a property of the engines and is
273 reported rather than normalised away, since normalising it would require
274 driving at least one engine in a manner its documentation does not describe.

Table 1: The three engines under test, each deriving the equations of motion by a different route, with the formulation pathways each exposes.

Engine	Route	Pathways exercised
MechanicsDSL 2.1.3 (a8dc2b2)	symbolic compiler; consumes a Lagrangian, direct solve for a fixed topology	Lagrangian, Hamiltonian, constrained
sympy.physics. mechanics 1.14.0	general-purpose computer algebra, driven as documented	Lagrangian, constrained
Drake 1.56.0	recursive articulated-body library for arbitrary topologies, contact, real-time control	model-building interface

275 3.2. The referee

276 Adjudication is performed by a closed-form reference derived by hand
277 and implemented in NumPy alone. This is the central design decision of the
278 study, and its rationale is straightforward: two implementations that share
279 a symbolic algebra layer, a linear solver, or a code-generation backend may
280 agree because they inherit the same defect, and their agreement then carries
281 no information about correctness.

282 **The independence criterion, stated exactly.** The reference shares no
283 symbolic algebra system, no mechanics library, no code-generation backend
284 and no derivation code with any engine under test. Every equation of motion
285 it evaluates was derived by hand and transcribed. This is a stronger criterion

286 than independent authorship, which permits two authors to import the same
287 package, and much stronger than independent implementation within one
288 ecosystem.

289 *The referee is corroborated from outside the study.* Adjudication by a sin-
290 gle reference concentrates risk: if it is wrong, every engine agreeing with it
291 is evidence of nothing. Its eleven self-tests do not dispose of this, since
292 self-tests share the author’s misconceptions — the objection Knight and
293 Leveson [14] raise against independent implementation. The reference was
294 therefore checked against a classical result predating the study: a uniform
295 chain of length L hanging under gravity has small-oscillation frequencies
296 $\omega_k = (z_k/2)\sqrt{g/L}$, where z_k is the k -th zero of J_0 . Linearising the reference’s
297 own mass matrix and potential and taking N to the continuum reproduces it:
298 relative errors on the first three modes fall from 5.5%, 4.3%, 1.6% at $N = 4$
299 to 0.19% on all three at $N = 128$, halving as N doubles. The reference
300 reproduces a result it was never fitted to.

301 It is not, and cannot be, total independence, and overstating it would be
302 the kind of claim this paper exists to scrutinise. The reference shares NumPy
303 for array arithmetic with the evaluation paths of all three engines; it shares
304 the pinned integrator (§3.4); and it shares IEEE-754 binary64 semantics and
305 the host processor. What it does not share is any code that decides *what* the
306 equations of motion are, which is the quantity under adjudication.

307 That distinction is load-bearing for §7, where a failure common to the
308 engines *and* the reference is reported. A reader is entitled to ask whether such
309 a failure is shared because the phenomenon is structural or merely because
310 the substrate is. §7.3 answers that question by measurement rather than by

311 assertion.

312 For the unconstrained families the reference solves the equations of motion
313 directly for a fixed topology. For the constrained families both systems admit
314 a constant diagonal mass matrix and holonomic constraints, so accelerations
315 and multipliers follow from the saddle-point system

$$\begin{bmatrix} M & J^\top \\ J & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} F(q) \\ -\dot{J}\dot{q} \end{bmatrix}, \quad (1)$$

316 in which both J and $\dot{J}\dot{q}$ have closed forms for the rod and circle constraints
317 involved. No symbolic algebra is required at any point in the reference, which
318 is what permits the claim of library independence to be made literally rather
319 than approximately.

320 The reference is deliberately left unstabilised: no Baumgarte term and
321 no projection. A stabilised reference would be solving different equations
322 from those the engines are asked for, and the resulting comparison would
323 be between two different problems. The cost of that choice is a constraint
324 drift of order 10^{-4} over three seconds, which does not affect the derivation
325 comparison because that comparison evaluates accelerations at given states.

326 3.3. *The case matrix and the freeze*

327 The case matrix comprises 55 cases over 36 systems across six axes: de-
328 grees of freedom, closed-loop topology, constraint redundancy, near-singular
329 inertia, mass ratio, and symbolic nesting depth. The matrix was frozen —
330 fixed in content and recorded with a version identifier — before the cross-
331 engine measurements reported here were taken.

332 The freeze is governed by an explicit rule. Extension of the harness is
333 permitted where it adds adjudication without revising existing verdicts, and
334 any new axis must be declared before it is measured. Removing a case after
335 observing its result is selection on the outcome and is forbidden. The rule
336 exists because the author maintains one of the engines under test, and the
337 temptation it guards against is therefore not hypothetical.

338 Of the 55 cases, 45 are adjudicated; the remainder exceed the wall-clock
339 limit for at least one engine and are recorded as timeouts rather than as
340 verdicts. A timeout is scored as a distinct category from a pass or a failure,
341 because an absence of an answer and a wrong answer are different events and
342 conflating them would obscure precisely the distinction this study exists to
343 examine.

344 3.4. Integrator pinning

345 Every engine supplies only its right-hand side. Integration is performed
346 by a single call to one explicit Runge–Kutta scheme (DOP853) with identical
347 relative and absolute tolerances ($\text{rtol} = 10^{-10}$, $\text{atol} = 10^{-12}$) for all engines.

348 Pinning has a consequence that must be stated plainly: it makes the
349 measurement structurally incapable of ranking the engines on integration
350 accuracy. Energy drift under a pinned integrator is dominated by the inte-
351 grator the engines share, not by the engines. This is not a limitation of the
352 design but its purpose. Identical drift across engines is evidence that their
353 equations are equivalent, which is what the comparison exists to establish.
354 Without pinning, one engine’s native integrator would have been compared
355 against another’s, and the difference attributed to the engines.

356 The design has a corresponding limitation, disclosed here rather than

357 in the results. An engine whose constrained dynamics are enforced inside
358 its own solver cannot participate in a pinned-integrator comparison on con-
359 strained systems without being asked to do something other than what it is
360 built to do. Where this arises, the affected engine is scored on constraint sat-
361 isfaction alone, and the asymmetry is reported rather than concealed. This is
362 the case for the recursive library on the slider–crank (§5.6); the same engine
363 participates fully on the cart–pole, which is a tree and requires no constraint.

364 3.5. Adjudication criteria

365 Two quantities are compared. The first is the residual between an engine’s
366 computed accelerations and the reference’s, evaluated at 16 probe states per
367 case drawn from the constraint manifold where one applies. The second
368 is energy drift over a fixed integration horizon under the pinned integrator.
369 For constrained cases a third quantity, constraint residual drift from $t = 0$,
370 is recorded. The adjudication tolerance throughout is 10^{-8} relative. In the
371 vocabulary of §2.3, the first quantity is the code-verification measurement
372 proper — it compares derived equations against an exact standard — while
373 the second and third are consistency checks on the integrated trajectory and
374 are not independent of the shared integrator.

375 Constraint residual is measured as drift from the initial value rather than
376 as an absolute quantity. The suite’s constrained initial conditions are rounded
377 to four decimal places, so every constrained case begins approximately 10^{-4}
378 off its own constraint manifold. This is harmless for the classifier as con-
379 structed, but a reader checking the absolute residual would find an apparent
380 violation and might reasonably attribute it to the engine. The fact is recorded
381 here so that it cannot mislead.

382 3.6. *Threats to validity*

383 *Common authorship of engine and referee.* The referee shares no derivation
384 code with any engine, but it does share an author with one of them. A
385 misconception held by that author could in principle appear in both, and
386 their agreement would then be uninformative. Two considerations bound
387 this risk. The two engines the author did not write agree with the reference
388 to the same tolerance, and a shared misconception would have to be present
389 in all four independently.

390 Separately, the study record documents **four** instances in which the au-
391 thor’s expectation about the *measuring apparatus* was contradicted by an
392 independent artefact, and corrected before the affected results were reported:
393 Instances 1–3 are convention mismatches in the comparison and are the basis
394 of the diagnostic in §9; instance 4 is a mistaken adjudication criterion. All
395 four concern the instrument, and all four were detected by consulting an
396 artefact rather than by reasoning further.

397 The study record contains further corrected errors that concern external
398 facts rather than the instrument — chiefly the provenance of a third-party
399 API guard, established only after documentation, a cached search result, and
400 an unvalidated repository query had each produced a different wrong answer.
401 Those are omitted here. They bear on the author’s diligence, not on whether
402 the measurement detects its own faults, and only the latter is evidence.

403 *Mechanism coverage.* Three mechanism families are examined. They were
404 chosen for structurally distinct degeneracies — a vanishing transmission ra-
405 tio, a collapsing mass-matrix determinant, and a chaotic regime reached by
406 amplitude — but they remain three families, and the results do not license

Table 2: Faults in the measuring apparatus, each contradicted by an independent artefact and corrected before the affected results were reported.

#	Mistaken expectation	Caught by
1	The Hamiltonian pathway's right-hand side returns accelerations; it returns $\dot{p}$	discriminating $\dot{q}$ against $M^{-1}p$
2	A revolute chain reports absolute joint angles; it reports relative ones	inspecting the mass matrix
3	A pole offset rotated about $+y$ matches a reference using $x + l \sin \theta$; it mirrors it	inspecting the mass matrix
4	An engine reporting motion at an exact equilibrium is wrong; the exact solution is unreachable in finite precision	the independent reference behaving identically

407 claims about mechanism classes outside them.

408 *A comparator that is not independent on one family.* On the cart–pole the
409 computer-algebra package’s residual against the reference is *exactly zero at*
410 *all twenty configurations*, not merely small. This is addressed in §5.6: on that
411 family the two are not independent, and the effective comparison is three-way
412 rather than four-way.

413 *Wall-clock timeouts.* The reach results rest on a wall clock measured on a
414 single machine in single runs, so the specific link counts at which an engine
415 ceases to return are machine-dependent and should be read as such.

416 *Driving style confounds capability comparison.* This threat was realised rather
417 than avoided, and is reported because it bears on how the reach results
418 should be read. Each engine was initially exercised along a single path, and
419 for the computer-algebra package that path was the one its documentation
420 prescribes, which routes through a symbolic linear solve whose cost grows ex-
421 plosively. The resulting ladder was read as a capability separation. Driving
422 each engine along the best path available through its own public interface re-
423 moves the separation entirely (§6). A comparison of engines on capability is
424 therefore a comparison of engines-as-driven, and the choice of path belongs
425 in the method rather than being left implicit. The correctness results are
426 not affected: they adjudicate each engine against an independent reference
427 rather than against each other.

428 *Non-portable axes.* One axis of the frozen matrix — symbolic nesting depth
429 — is meaningful only to an engine that consumes raw Lagrangians, and no

430 library-independent reference for it can exist. Those cases are excluded from
431 the cross-engine claim by construction rather than by choice.

432 *Three engines is a small sample.* It is, and for a statistical claim it would
433 be disqualifying. The claim made here is not statistical. **That differential**
434 **testing is blind to shared failures is established by exhibiting one,**
435 **not by estimating how often they occur:** a single failure common to
436 every implementation *and* to a referee sharing no derivation code with any of
437 them is sufficient to show the class is non-empty, and that adding implemen-
438 tations cannot empty it. The argument is an existence proof, and existence
439 proofs do not require samples.

440 The argument does not rest on this domain alone. A companion study [4]
441 repeats it in numerical quadrature, a setting with no mechanical content and
442 no shared code with anything measured here: five routines spanning three
443 algorithm families — QUADPACK through two language bindings, a tanh-
444 sinh rule, and two Simpson variants — integrate a narrow Gaussian bump
445 and return *bit-identical* wrong answers, with four of the five reporting an
446 estimated error of exactly zero. The mechanism there is different — adaptive
447 sampling misses a feature narrower than its initial grid, rather than round-off
448 amplified by an instability — but the structure is the same, and so is the
449 consequence: the implementations agree, and only a closed-form reference
450 detects the error. **Two unrelated domains, two unrelated mechanisms,**
451 **the same blindness.**

452 What these studies *cannot* support is the converse: nothing here estimates
453 how common shared failures are, in either domain or any other, nor what
454 fraction of defects differential testing misses in practice. Those are frequency

455 questions and would need a population. The negative result on claim (B) is
456 reported in the same spirit — as the absence of a disagreement across every
457 case measured, not as evidence that disagreements are rare.

458 *Constrained continuous-time dynamics in the recursive library.* Constrained
459 dynamics are not reachable through the continuous-time interface used else-
460 where in this study: in the pinned version a continuous-time **MultibodyPlant**
461 carrying a distance constraint refuses to evaluate time derivatives, in its
462 rigid and compliant forms alike. The library does provide a newer integrator,
463 CENIC, which does solve constrained continuous models and whose distance-
464 constraint support is present in the pinned version. It reaches them, however,
465 by building its own model and implementing stepping directly rather than
466 by exposing a right-hand side: what it returns are the positions and veloci-
467 ties resulting from a step. The plant’s generalised-acceleration output port
468 does not reflect the dynamics CENIC actually solved, and is not expected
469 to until the issue tracking constraint-aware reporting is closed; even then it
470 may report the trailing step rather than the instantaneous value. This study
471 adjudicates on acceleration residuals, so for a constrained continuous model
472 in the pinned version no acceleration field is available by any route. The
473 engine is therefore driven in its discrete mode on the slider–crank, where it
474 cannot share the pinned integrator and is scored on constraint satisfaction
475 alone.

476 This is the study’s design meeting a design decision in the integrator
477 rather than a defect in either, and the account above was confirmed by the

478 library’s maintainers in a public discussion.¹ An adjudication on positions
479 and velocities rather than accelerations could step that integrator directly
480 for a purely continuous system; that is a different protocol, and a separate
481 study.

482 3.7. Disclosure

483 The author maintains one of the three engines measured. The mitigation
484 is structural rather than declarative: no engine adjudicates itself or any other,
485 the referee shares no derivation code with any of them, the case matrix was
486 frozen before measurement under a rule forbidding post hoc removal, and
487 the results reported include the measurements on which the author’s engine
488 performs worst (§8.5).

489 AI assistance was used for portions of the implementation, the analysis,
490 and the drafting of parts of this manuscript; see the disclosure statement.

491 All figures reported were produced by executing committed code. Arte-
492 facts — the case matrix, the reference implementation, the adapters, the
493 sweep scripts, and the raw output records — are archived at [doi:10.5281/](https://doi.org/10.5281/zenodo.22315172)
494 [zenodo.22315172](https://doi.org/10.5281/zenodo.22315172) under the MIT licence.

495 4. Systems

496 Three mechanism families are examined. They were chosen so that each
497 presents a degeneracy arising from a different mechanism, on the reasoning

¹`RobotLocomotion/drake` discussion #24975, <https://github.com/RobotLocomotion/drake/discussions/24975>. Distance-constraint support in CENIC was added in #24776 and is included in v1.56.0; constraint-aware reporting of reaction forces and accelerations is tracked by #24061.

498 that a comparison exercising one kind of hard case repeatedly establishes less
499 than one exercising several kinds once.

500 *4.1. The planar N -link chain*

501 An open chain of N links under gravity, with revolute joints and no con-
502 straint. The chain scales in degrees of freedom without changing its structure,
503 which makes it the natural vehicle for the reach measurements of §6, and
504 its dynamics pass from regular to chaotic as the initial amplitude increases,
505 which makes it the vehicle for the amplitude axis.

506 The degeneracy here is dynamical rather than geometric. At the suite's
507 original initial angles the motion is quasi-periodic: perturbing the initial
508 state by 10^{-10} and integrating for ten seconds produces growth of order
509 10^0 . At larger amplitudes the same perturbation grows by eleven orders of
510 magnitude over the same horizon. Growth of order 10^0 is polynomial and
511 indicates regular motion; growth of order 10^{11} is exponential and indicates
512 chaos.

513 The amplitude axis was added after measurement showed that the frozen
514 suite had been exercising the chain only in its regular regime — that is, that
515 a suite whose stated method is to increase difficulty until behaviour breaks
516 down had been operating entirely within the well-behaved region of its princi-
517 pal system. The axis was declared before it was measured, in accordance with
518 the governance rule, and the Lagrangian it uses is character-for-character the
519 one already frozen; only the initial conditions differ.

520 4.2. The slider-crank

521 A crank rotating about a fixed pivot, connected by a rigid rod to a slider
 522 constrained to a straight line. The mechanism introduces two features absent
 523 from the chain: a closed kinematic loop, and a prismatic joint alongside a
 524 revolute one.

525 The constraint admits a closed form. Expanding $|S - P|^2 - l^2$ yields
 526 $x^2 - 2rx \cos \theta + r^2 - l^2$, the trigonometric identity eliminating the remaining
 527 terms, so no symbolic algebra is required to construct the reference.

528 The degeneracy is geometric and arises from the transmission ratio. The
 529 effective inertia

$$M_{\text{eff}}(\theta) = m_c r^2 + m_s \left(\frac{dx}{d\theta} \right)^2 \quad (2)$$

530 varies by four orders of magnitude over a single revolution when the slider
 531 is heavy, because $dx/d\theta$ vanishes exactly at $\theta = 0$ and $\theta = \pi$. At those two
 532 configurations — the dead centres — the slider ceases to contribute inertia
 533 altogether and the mechanism approaches the loss of its mass matrix. The
 534 measured variation is 1.12×10^2 at $m_s/m_c = 10^2$, 1.11×10^4 at 10^4 , and
 535 1.11×10^6 at 10^6 . The degeneracy is produced by the geometry rather than
 536 by a hand-constructed ill-conditioned matrix, which is what makes it a fair
 537 test.

538 Two parameters were swept: the rod-to-crank length ratio, from 3.0 down
 539 to 1.01 as the mechanism approaches its folding configuration, and the slider-
 540 to-crank mass ratio to 10^6 . Engines were evaluated both at random states
 541 on the constraint manifold and exactly at both dead centres.

542 4.3. *The cart–pole*

543 A cart translating on a horizontal prismatic joint with a pole suspended
544 from it by a revolute joint. Unlike the slider–crank the system is a tree, with
545 no loop and no constraint, which permits all three engines to be compared
546 in their ordinary modes of operation under a single pinned integrator, with
547 nothing excluded for reasons unrelated to dynamics.

548 The system is structurally new to the study in two respects: it combines
549 joint types within one mechanism, and its mass matrix is genuinely configuration
550 dependent, where the suite’s earlier multi-body systems had constant
551 ones.

552 Its degeneracy arises from a third mechanism again. The determinant

$$\det M(q) = ml^2 (M + m \sin^2 \theta) \quad (3)$$

553 collapses by six orders of magnitude twice per swing when the cart is light
554 and the pole passes through vertical, because a near-massless cart cannot
555 resist the pole’s horizontal reaction. The slider–crank degenerates because
556 a transmission ratio vanishes; the cart–pole degenerates because an inertia
557 vanishes. The causes are unrelated, which is the reason for testing both.

558 5. Results: agreement

559 5.1. *Equations of motion*

560 Residuals between each engine’s computed accelerations and the refer-
561 ence’s are constant across the amplitude axis, as expected: the equations of
562 motion do not depend on where the system starts.

Table 3: Equation-of-motion residuals against the reference, three-link chain, 16 probe states drawn uniformly from $[-0.5, 0.5]$. Relative to the double-precision unit round-off $u \approx 2.22 \times 10^{-16}$.

Engine	Residual	Multiple of u
MechanicsDSL	1.887×10^{-15}	8.5
Drake	1.072×10^{-14}	48
SymPy	2.620×10^{-14}	118

563 All three engines are therefore correct to the limit of the arithmetic, and
564 the differences between them reflect accumulated rounding over differing op-
565 eration counts rather than any difference in the equations derived.

566 5.2. Robustness of the probe sampling

567 Adjudication compares accelerations at probe states, so agreement could
568 in principle be an artefact of where those states were drawn. The comparison
569 was therefore repeated with five seeds and four sampling regimes chosen to
570 stress different mechanisms: the study’s own moderate box; velocities an
571 order of magnitude larger, where Coriolis terms dominate and any error in
572 the $C(q, \dot{q})\dot{q}$ bracket is amplified; angles clustered within 10^{-3} of π , where the
573 gravity terms nearly cancel; and nearly aligned angles, where a serial chain’s
574 mass matrix is worst-conditioned and the linear solve most fragile.

575 **No cell exceeds the 10^{-8} adjudication tolerance, and the worst dis-**
576 **agreement anywhere in 19,200 comparisons is 6.5×10^{-12} .** Agreement
577 is not an artefact of the sampling: it survives a tenfold increase in veloc-

Table 4: Worst relative disagreement with the reference over $5 \text{ seeds} \times 64 \text{ probe states per cell}$, each engine driven along its best available path. 19,200 comparisons in total.

N	Engine	uniform	wide	near- π	aligned
3	MechanicsDSL	1.0×10^{-14}	3.0×10^{-14}	2.9×10^{-16}	2.5×10^{-14}
	SymPy	8.0×10^{-15}	2.9×10^{-14}	1.3×10^{-16}	1.6×10^{-14}
	Drake	2.8×10^{-14}	4.1×10^{-14}	9.0×10^{-16}	8.1×10^{-14}
8	MechanicsDSL	1.9×10^{-14}	1.7×10^{-13}	1.7×10^{-16}	1.4×10^{-14}
	SymPy	3.4×10^{-14}	3.1×10^{-13}	1.7×10^{-15}	6.4×10^{-14}
	Drake	9.3×10^{-13}	6.1×10^{-13}	8.9×10^{-14}	1.6×10^{-12}
12	MechanicsDSL	5.2×10^{-14}	4.0×10^{-13}	3.3×10^{-16}	2.8×10^{-14}
	SymPy	9.4×10^{-14}	6.2×10^{-13}	7.6×10^{-15}	1.3×10^{-13}
	Drake	6.5×10^{-12}	2.8×10^{-12}	4.6×10^{-13}	5.0×10^{-12}

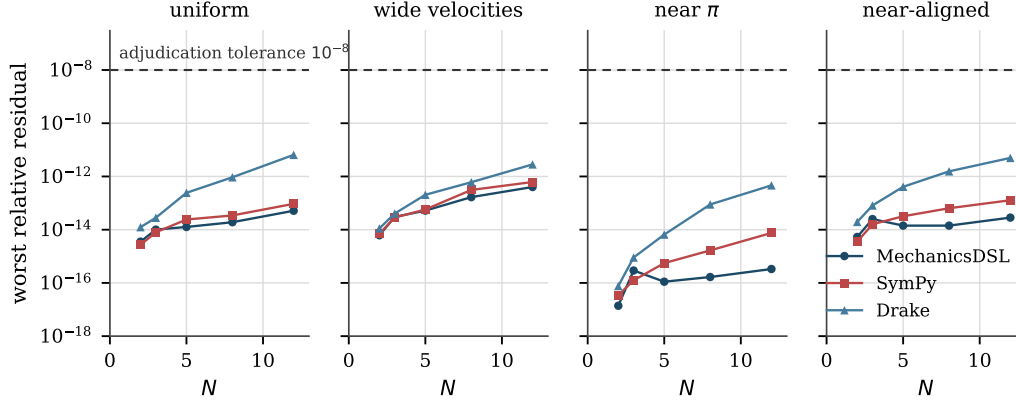


Figure 1: Agreement is not an artefact of the sampling. Worst relative residual against system size for each engine under four sampling regimes, over five seeds and 64 probe states per cell. The dashed line is the 10^{-8} adjudication tolerance; no point approaches it. The recursive library’s residuals are consistently larger and grow with N , as expected of a recursive numerical algorithm measured against a closed form. Data: Table 4.

ity range, the near-cancellation of gravity near π , and the worst-conditioned configurations a serial chain admits.

Two features of the table are worth noting rather than passing over. The near- π column shows the *smallest* residuals of the four regimes, because the accelerations there are themselves near zero and the comparison is relative; this is the regime in which §7 nonetheless finds every implementation failing, which is precisely the point that residual agreement cannot detect a shared failure. And the recursive library’s residuals are consistently one to two orders larger than the symbolic engines’, growing with N — expected of a recursive numerical algorithm against a closed form, and far inside tolerance at every point.

589 *5.3. The amplitude axis*

590 Across eight amplitudes spanning perturbation growth from single figures
 591 to 5.4×10^{11} , and three engines, 24 engine–amplitude pairs were evaluated.
 592 There were no refusals, no incorrect equations, and no integration failures.

Table 5: Amplitude axis, three-link chain. Perturbation growth is the factor by which a 10^{-10} initial displacement grows over 10s, measured on the reference at $\text{rtol} = 10^{-12}$. Energy drift is relative, under the pinned integrator.

Amp. (rad)	Growth	Regime	MechanicsDSL	SymPy	Drake
0.15	2.3	regular	7.302×10^{-11}	7.328×10^{-11}	7.445×10^{-11}
0.50	3.9	regular	2.201×10^{-11}	2.201×10^{-11}	2.201×10^{-11}
1.00	3.8	regular	2.702×10^{-11}	2.702×10^{-11}	5.123×10^{-11}
1.50	3.5×10^4	chaotic	1.723×10^{-10}	1.723×10^{-10}	2.860×10^{-10}
2.00	1.1×10^7	chaotic	2.137×10^{-10}	2.137×10^{-10}	2.139×10^{-10}
2.50	2.8×10^{10}	chaotic	4.737×10^{-10}	4.767×10^{-10}	7.783×10^{-10}
3.00	1.7×10^{11}	chaotic	3.036×10^{-10}	3.036×10^{-10}	3.042×10^{-10}
π	5.4×10^{11}	chaotic	2.737×10^{-10}	2.737×10^{-10}	2.737×10^{-10}

593 Energy drift rises with amplitude, as harder dynamics demand more of
 594 the integrator, and does so almost identically for all three engines: at the
 595 worst case the three values are 4.74, 4.77 and 7.78×10^{-10} , the same quantity
 596 to within a factor of 1.6.

597 *5.4. The chaotic regime*

598 At three degrees of freedom with all links started well into the chaotic
 599 regime, energy drift is indistinguishable across the reference and all three

engines.

Table 6: Chaotic regime, three-link chain, all links started at 3.0 rad. Perturbation growth 1.26×10^{11} over 10 s at $\text{rtol} = 10^{-10}$, implying a Lyapunov rate $\lambda = 2.556 \text{ s}^{-1}$. Divergence is the time at which the trajectory departs the reference’s by 10^{-6} ; §8.3 explains why it is not a ranking.

Engine	Energy drift	Divergence
reference	3.0358×10^{-10}	—
MechanicsDSL	3.0356×10^{-10}	6.96 s
SymPy	3.0361×10^{-10}	6.28 s
Drake	3.0421×10^{-10}	5.69 s

The largest separation is 0.2 per cent relative, twelve orders of magnitude below any physically meaningful quantity.

5.5. The constrained pathway

MechanicsDSL agrees with the closed-form reference of equation (1) on all eight constrained cases in the adjudicated set. It is the only engine scored here: the other two expose no interface for imposing holonomic constraints on a coordinate set supplied directly, so this pathway is single-engine by construction. Their constrained behaviour is measured instead on the slider–crank (§5.6), where the constraint is intrinsic to the mechanism and each engine can impose it in its own idiom.

The redundancy family is the more informative half. Its constraint Jacobian retains rank one while its row count grows to nine, so the multipliers

Table 7: Constrained pathway, MechanicsDSL against the saddle-point reference, 16 probe states per case. **redund_Rk** carries $k + 1$ constraints on two coordinates, all scalar multiples of one another.

Case	Coordinates	Constraints	Residual
loops_N3	4	3	5.788×10^{-16}
redund_R0	2	1	1.958×10^{-16}
redund_R1	2	2	1.084×10^{-15}
redund_R2	2	3	1.234×10^{-14}
redund_R3	2	4	2.554×10^{-15}
redund_R4	2	5	2.004×10^{-15}
redund_R6	2	7	3.266×10^{-15}
redund_R8	2	9	7.230×10^{-15}

613 become underdetermined while the accelerations remain unique. The engine
614 tracks the reference throughout. An exactly redundant constraint set does
615 not determine how the constraint force is distributed, only its total, and
616 neither the engine nor the reference is troubled by this.

617 *The Hamiltonian pathway.* The canonical route is the thinnest part of the
618 study’s coverage: the frozen ladder reached three coordinates before walling,
619 and until now the pathway had no independent check at all, only energy
620 conservation. It was therefore adjudicated directly in canonical coordinates,
621 comparing the engine’s $\dot{p}$ against the reference’s own Legendre transform
622 $p = M(q)\dot{q}$ at 64 probe states per size. The pathway agrees where it is

623 reachable — 0 at one coordinate and 2.7×10^{-15} at two — with the transform
 624 self-consistent to 1.1×10^{-16} , and walls at three coordinates as the frozen
 625 ladder records. No disagreement was found on the pathway most likely to
 626 harbour one.

627 The first attempt at this measurement reported a disagreement of 0.517
 628 at two coordinates. It was instance 1 of Table 2 a fourth time: the engine’s
 629 canonical right-hand side is a function of (q, p) and had been given $(q, \dot{q})$.
 630 The error vanished exactly at one coordinate, where $M = I$ makes the two
 631 coincide, which is the second tell described in §9 and the one that applies
 632 when the magnitude is not a clean 1 or 2. It is recorded here because it is
 633 the same fault recurring in the same place under a new harness, which is
 634 evidence about how easily the convention is confused rather than about any
 635 engine.

636 *Coverage.* With these cases adjudicated, the number of adjudicated cases
 637 carrying an independent derivation check rises from **22 to 40** of 45. The
 638 arithmetic does not compose as a simple increment, and the reconciliation is
 639 given in Table 8 because the figure is citable.

640 The five cases that remain unchecked are the Hamiltonian half of the symbolic
 641 axis. No library-independent reference for that axis can exist: nested cosine
 642 depth is meaningful only to an engine that consumes a raw Lagrangian, so
 643 there is no system to derive independently.

644 5.6. *The slider–crank and the cart–pole*

645 Across both additional families, 72 engine–case pairs were evaluated with
 646 no disagreements and no refusals.

Table 8: Coverage reconciliation over the 45 adjudicated cases. The two referees cover overlapping but different sets, so the increment is not additive.

Referee	Cases	Composition
Computer-algebra oracle (original)	22	Lagrangian pathway only, including 5 symbolic-axis cases
Library-independent reference	35	dof 7, mass_ratio 14, redundancy 7, near_singular 6, loops 1
Union: any independent check	40	
gained by the reference	18	
reachable only by the original oracle	5	symbolic-axis Lagrangian cases
Carrying no independent check	5	symbolic-axis Hamiltonian cases

647 All three engines agree with the reference at every mass ratio and every
648 starting angle, including where the determinant collapses by six orders of
649 magnitude, and energy drift is of order 10^{-9} or better throughout.

Table 9: Cart–pole, five cart-to-pole mass ratios $\times$ four initial angles $\times$ three engines. Residuals are constant across the four angles at each mass ratio. Energy drift is the worst over the four angles.

M/m	$\det M$ range	Residual			
		MechanicsDSL	Drake	SymPy	Worst drift
10^1	1.1	3.72×10^{-16}	2.78×10^{-17}	0	1.99×10^{-11}
10^0	2.0	3.72×10^{-16}	1.86×10^{-16}	0	3.75×10^{-10}
10^{-2}	10^2	5.02×10^{-15}	2.05×10^{-16}	0	1.60×10^{-9}
10^{-4}	10^4	7.49×10^{-15}	5.55×10^{-17}	0	1.11×10^{-9}
10^{-6}	10^6	1.25×10^{-14}	1.96×10^{-16}	0	3.36×10^{-9}

650 **The computer-algebra package is not an independent comparator**
651 **on this family.** Its residual against the reference is *exactly* zero — not small
652 — at all twenty configurations, over all sixteen probe states each. A residual
653 of exactly zero under a relative-error metric means the two implementations
654 produced bit-identical floating-point results, which indicates that they exe-
655 cuted the same arithmetic in the same order rather than that they agree by
656 independent derivation. The likely cause is that the cart–pole’s mass matrix
657 and forcing vector are simple enough that the package’s symbolic simplifica-
658 tion reproduces the hand-derived closed form exactly, and its code generation
659 then emits the same sequence of operations.

660 Consequently the cart–pole comparison is effectively three-way — refer-

661 ence, symbolic compiler, recursive library — not four-way. This does not
 662 affect the other two families, where the same package returns non-zero resid-
 663 uals (2.62×10^{-14} on the chain, 5.55×10^{-16} to 1.96×10^{-15} on the slider–crank)
 664 and is a genuine independent comparator.

665 Subject to that qualification, the recursive library returns the smallest resid-
 666 uals on this family, of order 10^{-16} — better than the symbolic compiler by
 667 two orders of magnitude. This is the expected outcome when an algorithm
 668 designed for arbitrary topologies is applied to a two-body tree, and it is the
 669 converse of the pattern observed on the longer chains, where its residuals are
 670 the largest.

671 Both symbolic engines hold the rod length to approximately 10^{-9} through
 672 dead-centre crossings at a length ratio of 1.05 and a mass ratio of 10^4 . The
 673 recursive library holds the rod to approximately 10^{-6} in its discrete mode.

674 5.7. Summary

675 No engine returned a wrong answer on any adjudicated case, in any family,
 676 in either dynamical regime, on either the constrained or the unconstrained
 677 pathway. Claim (B) of §1.3 — that engines differ in whether they report
 678 success on incorrect physics — is unsupported by every measurement in this
 679 study.

680 6. Results: reach, and what it measured

681 Agreement on cases all three engines can answer leaves open whether
 682 they can answer the same cases at all. Each engine was therefore taken up a
 683 ladder of system sizes to 30 links, with each engine–pathway–size combination
 684 built in its own subprocess under a fixed 120 s wall-clock limit, and the time

Table 10: Slider–crank. Residual and energy drift under the pinned integrator for the two engines that can share it; the recursive library is driven in its discrete mode and scored on constraint satisfaction alone (§3.4). Constraint column is the worst rod-length residual over a 5 s integration.

l/r	m_s/m_c	Engine	Residual	Energy drift	Constraint
3	10^2	MechanicsDSL	1.22×10^{-15}	1.51×10^{-10}	1.81×10^{-10}
3	10^2	SymPy	5.55×10^{-16}	1.51×10^{-10}	1.81×10^{-10}
3	10^2	Drake (disc.)	—	—	1.77×10^{-6}
1.5	10^2	MechanicsDSL	4.17×10^{-15}	2.11×10^{-10}	9.12×10^{-10}
1.5	10^2	SymPy	7.53×10^{-16}	2.12×10^{-10}	9.12×10^{-10}
1.5	10^2	Drake (disc.)	—	—	1.25×10^{-7}
1.05	10^2	MechanicsDSL	5.66×10^{-15}	5.40×10^{-10}	1.40×10^{-9}
1.05	10^2	SymPy	1.96×10^{-15}	5.40×10^{-10}	1.40×10^{-9}
1.05	10^2	Drake (disc.)	—	—	1.42×10^{-7}
3	10^4	MechanicsDSL	1.97×10^{-15}	2.92×10^{-10}	8.15×10^{-10}
3	10^4	SymPy	5.55×10^{-16}	2.90×10^{-10}	8.11×10^{-10}
3	10^4	Drake (disc.)	—	—	1.14×10^{-6}

685 measured to produce the equations of motion and evaluate the accelerations
686 once. Results were checked against the reference wherever an answer was
687 returned.

Table 11: Scaling ladder, Lagrangian pathway, 120 s wall clock per engine-size pair. Single runs on one machine. Times include model construction and one acceleration evaluation. Once an engine fails to return it is not retried at larger sizes.

	MechanicsDSL		SymPy		Drake	
N	time	residual	time	residual	time	residual
1	1.57 s	0	0.33 s	0	0.44 s	0
2	2.17 s	1×10^{-16}	0.38 s	4×10^{-16}	0.24 s	2×10^{-16}
3	21.63 s	4×10^{-16}	0.59 s	8×10^{-15}	0.25 s	6×10^{-15}
5	7.45 s	2×10^{-16}	2.58 s	1×10^{-15}	0.24 s	7×10^{-14}
8	42.03 s	2×10^{-15}	<i>timeout</i>		0.22 s	5×10^{-14}
12	64.43 s	6×10^{-15}	—		0.22 s	1×10^{-12}
20	<i>timeout</i>		—		0.23 s	1×10^{-11}
30	—		—		0.23 s	6×10^{-11}

688 Read on its own, this ladder appears to show three distinct capability
689 regimes: 5 links, 12 links, and 30, the largest size tested. §6.1 shows that
690 reading to be wrong. Each engine here was driven along a single path, and
691 for the computer-algebra package that path was the one its documentation
692 prescribes, which routes through a symbolic linear solve whose cost grows

693 explosively in the number of coordinates. The ladder measures those choices
694 as much as it measures the engines.

695 Two observations from it survive that correction.

696 The recursive library is in a different class of behaviour. Its cost is flat
697 at approximately 0.23 s from 5 links to 30, an $O(N)$ articulated-body algo-
698 rithm set against two symbolic approaches whose cost grows explosively. Its
699 residual grows with system size, from 6×10^{-15} at 3 links to 6×10^{-11} at 30,
700 remaining four orders of magnitude inside the adjudication tolerance.

701 Cost is not monotonic in system size for the symbolic compiler: three links
702 cost 21.63 s against 7.45 s for five. The effect is attributable to a threshold
703 in the engine’s symbolic-to-numeric handling and is reported rather than
704 smoothed.

705 The finding that matters is what did not occur. Every engine that re-
706 turned an answer on this ladder returned a correct one, and every engine
707 that could not, failed by never returning. A timeout is the loud failure mode:
708 the researcher knows the computation did not finish. No engine was observed
709 to return a wrong answer at any size measured, and the study’s original hy-
710 pothesis remains unsupported on this axis as on every other.

711 *6.1. The same systems, the same budget*

712 The ladder above gave each engine one path. Giving each engine the best
713 path available through its own public interface — Kane’s formulation with
714 a numeric solve and common-subexpression elimination for the computer-
715 algebra package, simplification disabled for the symbolic compiler, and the
716 single path the recursive library exposes — and asking all three for the same
717 systems under the same wall clock in one session gives a different picture.

Each cell here is adjudicated at six probe states rather than the sixty-four used in §5.2. The reduction is a cost decision, not a change of standard: building the symbolic equations for an 80-coordinate chain takes minutes, and each probe then evaluates a dense 80×80 mass matrix and solves it. Six states suffice here because this table asks whether an engine *answers at all* at a given size; the question of whether agreement survives adversarial sampling is the separate one settled in §5.2, at sizes where many more probes are affordable.

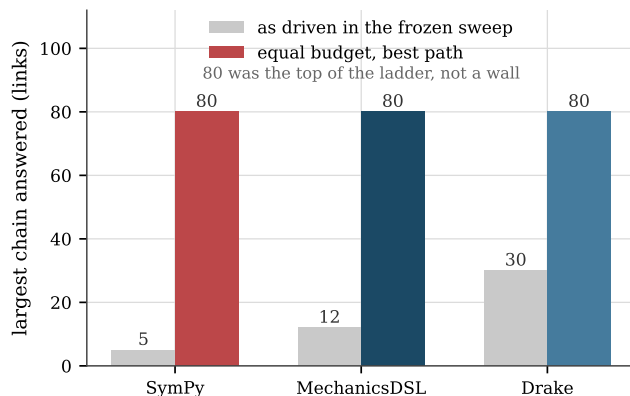


Figure 2: The capability separation dissolves under equal treatment. Grey bars are the frozen ladder, in which each engine was driven along a single path; coloured bars give the largest system each answers when driven along the best path through its own public interface under a common wall clock. Data: Table 11 and Table 12.

All three engines answer every rung to 80 coordinates, with no failures and no disagreements. The apparent capability separation of Table 11 does not survive equal treatment; 80 is where the ladder stopped, not where any engine failed.

What separates the engines is speed. The recursive library is flat at 0.2–

Table 12: Equal-budget reach. Each engine driven along the best path available through its public interface, same systems, same 5400 s wall clock, one session on one machine, adjudicated against the same reference at 6 probe states. Residual is the worst relative disagreement with the reference.

	MechanicsDSL		SymPy		Drake	
N	time	residual	time	residual	time	residual
40	145.0 s	1.7×10^{-13}	74.3 s	3.6×10^{-13}	0.3 s	1.7×10^{-10}
60	468.0 s	4.2×10^{-13}	280.2 s	7.1×10^{-13}	0.2 s	4.1×10^{-10}
80	1100.9 s	5.9×10^{-13}	693.3 s	1.2×10^{-12}	0.2 s	9.9×10^{-10}

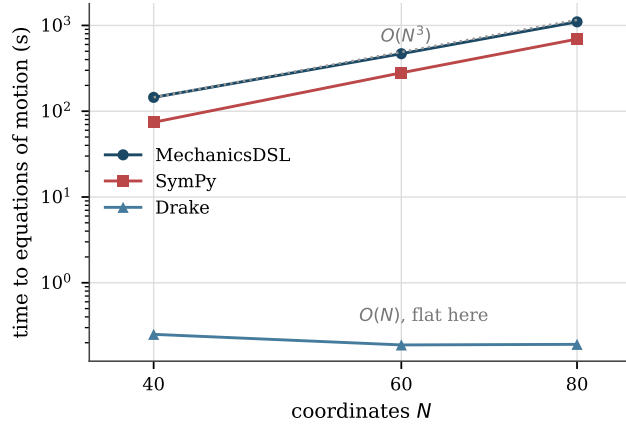


Figure 3: What separates the engines is cost, not reach. Time to produce the equations of motion against coordinate count, all three engines driven along their best available path. The two symbolic engines grow as roughly $O(N^3)$ (dotted guide); the recursive articulated-body library is flat across the same range, a difference of algorithm class rather than of engineering quality. Data: Table 12.

0.3 s across the ladder and at 80 coordinates outpaces the computer-algebra package by roughly $3600\times$ and the symbolic compiler by roughly $5800\times$ — a difference of algorithm class, not of engineering quality, and one that no change of driving affects. Between the two symbolic engines the computer-algebra package driven well is between $1.6\times$ and $2.0\times$ faster across the ladder, while the symbolic compiler returns residuals about twice as small on this family. The residual advantage is family-specific: it reverses on the cart-pole and the slider-crank, and is reported here only to be qualified. Neither difference is a capability boundary, and every value involved is orders of magnitude inside the adjudication tolerance.

The generalisable point is not about these three engines. It is that a comparison of engines on capability is a comparison of engines-as-driven, and that the path taken through each tool belongs in the method rather than being left implicit. The confound here survived a pre-registered case matrix and a frozen baseline, neither of which was designed to catch it.

7. A numerically induced departure from an exact unstable equilibrium, shared by every implementation

7.1. The configuration

Consider the N -link chain with every joint angle set to π and every velocity set to zero: the chain stands inverted, each link balanced directly above the one below. The exact solution is that nothing moves. Every term in the equations of motion vanishes analytically. The gravity term is proportional to $\sin \pi$, which is zero. The Coriolis term vanishes because the velocities are zero and the angles equal. The configuration is an equilibrium, and the exact

755 trajectory is the constant function.

756 It is, of course, an *unstable* equilibrium. That distinction is the whole of
757 what follows.

758 *What is and is not claimed here.* An objection presents itself immediately
759 and should be answered before the result rather than after it. A physical
760 pendulum released at the inverted position falls, so returning a large tra-
761 jectory might be called right rather than wrong. The objection concerns a
762 different system from the one under test. The engines were not asked what
763 a physical pendulum does; they were asked to integrate a stated initial-value
764 problem, and for that problem — exact angles of π , exact zero velocities —
765 the exact solution is the constant function. Every engine returns something
766 else.

767 The claim is therefore not that the engines are defective, nor that the re-
768 turned trajectory is unphysical. It is narrower and, for a user, worse: **every**
769 **implementation returns an answer dominated by its own round-off,**
770 **reports success, and gives no indication that it has done so.** A
771 researcher who did not already know the initial condition was an unstable
772 equilibrium would have no way of learning it from the output. That is the
773 failure mode the study set out to look for — a wrong answer returned with
774 every indication of success — and it is the one case in which it was found.
775 Because its cause lies in the arithmetic rather than in a defect in any imple-
776 mentation, it is described here as a numerically induced departure from the
777 exact solution rather than as a fault in any engine.

778 7.2. *The observation*

779 All three engines, and the independent reference, return approximately
780 20 radians of motion over a ten-second horizon. All four report success. None
781 emits a warning, a diagnostic, or any indication that the returned trajectory
782 bears no relation to the exact solution of the system as specified.

783 Energy is conserved throughout to 2.737×10^{-10} — the same tolerance as
784 in every other case in the study, and identical across all three engines (Table 5,
785 final row). The trajectory is smooth, physically plausible in isolation, and
786 entirely wrong. **That energy is conserved so well during the fall is**
787 **precisely why every check in the suite passes it.**

788 7.3. *The mechanism*

789 The cause is arithmetic rather than algorithmic. In binary64, $\sin(\pi)$ evalu-
790 ates not to zero but to 1.2246×10^{-16} , because π is not exactly representable.²
791 This leaves a residual gravitational acceleration of 1.20×10^{-15} at the initial
792 state. The equilibrium being unstable, that residual is amplified exponen-
793 tially, and within the ten-second horizon it reaches full scale.

794 This raises an alternative the argument so far does not exclude. Every
795 engine and the reference ran at the same precision on the same hardware,
796 so the failure might be shared because the substrate is shared rather than
797 because the phenomenon is structural. The two predict different things and

²Table 13 reports a smaller residual, 1.14×10^{-17} , in its first row. The two are not in conflict: that row uses arbitrary-precision arithmetic at 16 *decimal* digits, whereas binary64 carries 53 bits, or about 15.95 decimal digits. The residual scales with the working precision, so a slightly wider mantissa yields a slightly smaller ε . Nothing in the argument depends on which is used.

798 the difference is measurable: if the failure is an artefact of binary64, sufficient
799 precision removes it; if it is structural, precision only delays it.

800 Integrating the perturbation in arbitrary precision settles it (Table 13).
801 **That table measures a different system from the one above, and**
802 **deliberately so.** It uses a single linearised coordinate — an inverted simple
803 pendulum — rather than the multi-link chain of §7.2, because the question
804 there is whether precision removes the failure, and a one-coordinate system
805 isolates that from the chain’s coupling and chaos. The rates therefore differ:
806 the chain reaches roughly 20 rad inside ten seconds, while the pendulum takes
807 11.9 s to reach 0.1 rad at binary64 precision. Only the scaling with precision
808 is being compared, not the times themselves.

809 The linearisation deserves the same treatment, since a result obtained
810 from $\ddot{\varphi} = \lambda^2 \varphi$ is a result about that equation and not about the chain. Here
811 the approximation is unusually safe: linearising $\sin \varphi \approx \varphi$ incurs a relative er-
812 ror of order $\varphi^2/6$, and the perturbations in question run from 10^{-17} down to
813 10^{-151} . The linearisation is therefore exact to between thirty and three hun-
814 dred significant figures over the whole interval that sets the divergence time,
815 and the trajectory has left that regime only in its final moments, long after
816 the scaling has been determined. Nothing in the measured slope depends on
817 the nonlinear terms. Linearising about the equilibrium gives $\ddot{\varphi} = \lambda^2 \varphi$ with
818 $\lambda = \sqrt{g/l} = 3.1321 \text{ s}^{-1}$, so a residual $\varepsilon \sim 10^{-p}$ at p working digits should
819 reach a fixed threshold after a time linear in p , with slope $\ln 10/\lambda = 0.735 \text{ s}$
820 per digit. That is what is observed, to the integration step, across nine orders
821 of magnitude in precision.

822 The measured slope is 0.735 s per digit against a predicted 0.735, the relation

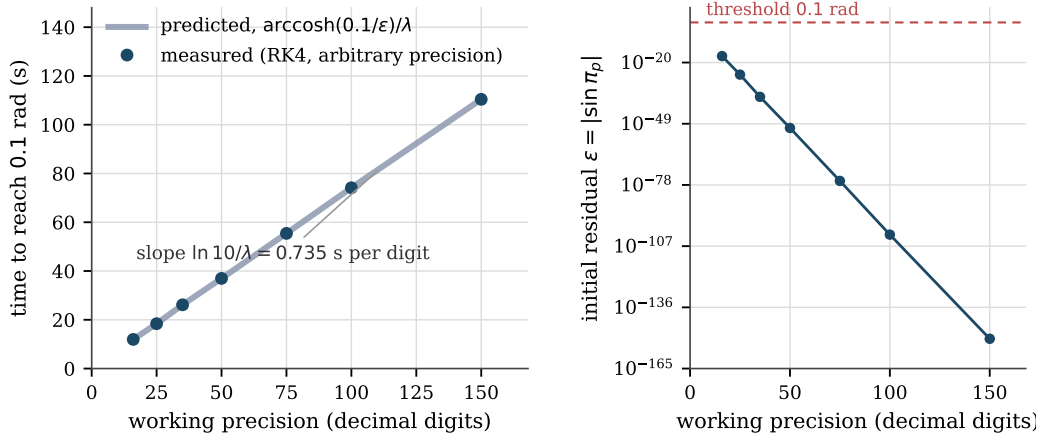


Figure 4: Precision delays the shared failure and never removes it. **Left:** time for the perturbation to reach 0.1 rad against working precision. Measurement (points) and the closed-form prediction $\text{arccosh}(0.1/\epsilon)/\lambda$ (line) coincide to the integration step at every precision; the relation is linear with slope $\ln 10/\lambda = 0.735$ s per digit. **Right:** the residual $\epsilon = |\sin \pi_p|$ that seeds the divergence, falling nine orders of magnitude across the same range and never reaching zero. Data: Table 13.

is monotone in precision, and **at no precision tested does the failure disappear**. Precision buys delay proportional to the number of digits and never buys immunity. **No computation that evaluates the equations of motion in finite-precision floating point can remain at an unstable equilibrium**, and the behaviour is a property of the problem in that arithmetic rather than a defect in any engine.

The qualifier is not decoration. A computation carrying π symbolically, or working in exact rational arithmetic with the gravity term evaluated in closed form, obtains exactly zero and stays put indefinitely; the failure reported here is a property of floating-point *evaluation*, not of computation

Table 13: Divergence from the inverted equilibrium against working precision. Fixed-step RK4 in arbitrary precision, perturbation coordinates, threshold 0.1 rad. Predicted time is $\operatorname{arccosh}(0.1/\varepsilon)/\lambda$.

Digits	$\varepsilon = \sin \pi_p $	Measured	Predicted
16	1.14×10^{-17}	11.94 s	11.94 s
25	1.88×10^{-26}	18.40 s	18.40 s
35	5.51×10^{-37}	26.14 s	26.14 s
50	1.01×10^{-51}	36.98 s	36.98 s
75	8.01×10^{-77}	55.43 s	55.43 s
100	2.87×10^{-102}	74.14 s	74.14 s
150	1.32×10^{-151}	110.41 s	110.41 s

in general. That distinction also marks the boundary of the practical claim: every engine examined here, and every engine in ordinary scientific use, evaluates its equations of motion in floating point. That the independent reference — which shares no code with any engine and was derived by hand — exhibits the identical behaviour is what establishes this. Had the reference remained at rest, the finding would have been a defect in three engines. It did not, and the finding is something else.

7.4. Why the case is retained

An earlier version of this axis was written on the expectation that an engine reporting motion at the inverted equilibrium is wrong and an engine reporting no motion is right. That expectation was false. It is recorded here,

844 and corrected in the module rather than silently revised, because the ease
845 with which it was held is part of the finding: the intuition that a correct
846 implementation should return the exact answer at an exact equilibrium is
847 natural, widely shared, and incorrect. It is instance 4 of Table 2.

848 7.5. *The same failure in a second mechanism*

849 One instance of a shared failure is a curiosity; the claim that differential
850 testing is blind to a *class* of failure requires more. The cart–pole supplies a
851 structurally different test: a tree rather than a chain, mixed joint types, and
852 an inverted equilibrium in which the cart is free to recoil, so the instability
853 couples two coordinates rather than one.

854 Started with the pole vertical and everything at rest, at cart-to-pole mass
855 ratios of 10, 1 and 10^{-2} , **all four participants — the three engines and**
856 **the reference — report success and return motion**, and do so in
857 agreement with one another to every digit displayed: 2.803×10^{-3} rad at
858 $M/m = 10$, 6.002 rad at $M/m = 1$, 6.283 rad at $M/m = 10^{-2}$. Not one of
859 the twelve cases stayed at rest, and not one warned.

860 The phenomenon is therefore not a property of the chain. Four indepen-
861 dent implementations agree closely on a trajectory that is entirely round-off,
862 in two mechanisms whose only common feature is that the initial condition
863 is an unstable equilibrium.

864 What the case exposes is real and shared. Every implementation tested
865 returns success and hands back a large trajectory where the exact answer is
866 no motion at all, and none warns that the initial condition is an unstable
867 equilibrium whose evolution is dominated by round-off. This is the study’s
868 original thesis appearing as a universal blind spot rather than as a difference

869 between engines. It is a weaker result than a disagreement would have been,
870 and an honest one.

871 *7.6. What follows for differential testing*

872 Differential testing rests on the premise that independent implementa-
873 tions fail independently, so that disagreement localises incorrectness. This
874 case is a counterexample of a specific kind. The failure is not independent
875 across implementations because its cause is not in any implementation: it is
876 in the interaction between the problem and the arithmetic every implemen-
877 tation shares.

878 Cross-implementation comparison cannot detect a failure of this kind,
879 because every implementation exhibits it identically. Adding a fourth engine,
880 a fifth, or a reference derived from first principles by an independent author
881 would not help — this study added exactly such a reference, and it failed in
882 the same way.

883 The class of failures invisible to differential testing is therefore not empty,
884 and it is not exotic. Unstable equilibria are not pathological configurations
885 contrived for a test suite; they are the operating point of any inverted-
886 pendulum control problem and appear throughout robotics and biomechanics.
887 A method that cannot see them has a blind spot in a region practitioners
888 actually work in.

889 **8. What these measurements do and do not support**

890 *8.1. The integrator was pinned, which removes the variable*

891 Every engine supplied only its right-hand side, and integration was a sin-
892 gle call with identical tolerances for all of them. Energy drift is therefore

dominated by the shared integrator, not by the engines. Identical drift is evidence that the equations are equivalent, which is what pinning was intended to establish. The measurement is structurally incapable of ranking the engines on integration accuracy, and that was its purpose.

8.2. *The differences are not differences*

The worst energy-drift values across the amplitude axis are 4.74, 4.77 and 7.78×10^{-10} — the same quantity to within a factor of 1.6. In the chaotic case the three values differ by 0.2 per cent relative, twelve orders of magnitude below any physically meaningful quantity. No ranking can be built on separations of this size.

8.3. *Divergence time cannot rank the engines*

In a chaotic system the time at which one trajectory departs from another is governed by the size of the initial difference between their right-hand sides, amplified exponentially. With the Lyapunov rate $\lambda = 2.556 \text{ s}^{-1}$ implied by the measured growth, a seed s should cross a 10^{-6} threshold at $t = \lambda^{-1} \ln(10^{-6}/s)$.

The band is predicted; the ordering is not. Predicted times span 6.8–7.9 s and observed times span 5.7–7.0 s, so the mechanism is right — but the predicted order is MechanicsDSL, Drake, SymPy while the observed order is MechanicsDSL, SymPy, Drake. Drake and SymPy transpose.

The reason the ordering fails is instructive, and it strengthens rather than weakens the conclusion. The three seeds span barely one order of magnitude while the amplification spans eleven. The effective seed is also the residual *along the actual trajectory*, not the maximum over random probe states, and

Table 14: Divergence time against the prediction from each engine’s own right-hand-side residual, three-link chain in the chaotic regime. The band is predicted; the ordering is not.

Engine	$N=3$ residual	Predicted	Observed
MechanicsDSL	1.89×10^{-15}	7.86 s	6.96 s
Drake	1.07×10^{-14}	7.18 s	5.69 s
SymPy	2.62×10^{-14}	6.83 s	6.28 s

the threshold crossing is itself noisy in a chaotic system. Divergence time therefore has no resolving power between engines whose equations agree this closely: it measures the Lyapunov exponent, with the engine identity as a rounding error.

8.4. *Why one engine’s residual is larger, and why that is not worse*

The recursive library evaluates dynamics with an articulated-body algorithm built for arbitrary topologies, contact, and real-time control. The reference and the symbolic engines perform a direct solve of one hand-derived Lagrangian for one fixed topology. More arithmetic operations accumulate more rounding. At 7×10^{-14} on the longer chains this is a deliberate generality-for-precision trade, not a defect, and it remains six orders of magnitude inside the adjudication tolerance. The cart–pole results of Table 9, where the same engine returns the smallest residuals of the three, confirm the interpretation.

930 *8.5. Where the engines genuinely differ, the author’s engine is behind*

931 Compile time at three links is approximately 37 times slower than the
932 computer-algebra package (21.63 s against 0.59 s, Table 11). The Hamilto-
933 nian pathway times out from three links, where the other engines do not. On
934 the near-singular axis its accuracy is 5.0×10^{-12} against 1.7×10^{-15} . On the
935 cart–pole its residuals are two orders of magnitude larger than the recursive
936 library’s. These measurements are reported for the same reason as the rest:
937 a study that reported only the comparisons its author’s engine wins would
938 not be worth reading.

939 *8.6. What can be claimed*

940 That the three engines agree with an independent closed-form derivation,
941 and with one another, to within 3×10^{-14} on equations of motion, across
942 amplitudes spanning the regular and chaotic regimes, across three mechanism
943 families, and across constrained and unconstrained pathways.

944 Energy drift is indistinguishable across the engines as well, but that is
945 a consistency check rather than a second independent result: all four share
946 the pinned integrator (§3.4), so near-identical drift is close to guaranteed
947 by construction. The claim rests on the acceleration residuals, which are
948 computed independently on each side.

949 The bound is set by the largest residual anywhere in the study — the
950 computer-algebra package at 2.62×10^{-14} on the three-link chain (Table 3)
951 — not by the smallest. It is stated as 3×10^{-14} rather than more tightly
952 because a bound that does not cover every measurement is not a bound.

953 That is an independently adjudicated correctness result. A claim of su-
954 periority for any one engine would rest on differences of a fraction of a per

cent in a metric governed by a shared integrator, and — in the case of the author’s engine — would be advanced by the party with an interest in it. Such a claim would be identified immediately and would discredit the work that is sound.

9. A diagnostic for convention mismatch

Three times during this study a comparison produced an error whose pattern across a sweep, rather than its magnitude, revealed the cause. On each occasion that cause was a convention mismatch between the reference and the engine adapter, not a defect in the engine. Two of the three produced an error identical across every case to many significant figures; the third did not, and the difference between them turns out to matter.

Table 15: The three convention mismatches encountered in this study, the error each produced, and the degenerate configuration in which it was invisible.

Observed error	Cause	Degenerate case that hid it
4.53, and exactly 1.00 at dead centre	Hamiltonian pathway integrates (q, p) and returns $\dot{p}$; compared against $\ddot{q}$	$N=1$, where $M = I$ makes $p = \dot{q}$
2.5–6.4, growing in N	engine reports <i>relative</i> joint angles; reference uses <i>absolute</i>	$N=1$, where the two coincide
exactly 2.00, all 20 cases	pole offset rotated about $+y$ mirrors the mechanism, flipping the cart’s acceleration	—

966 A discrepancy arising from genuine numerical difference varies with con-
 967 figuration, because the quantities producing it vary with configuration. A
 968 discrepancy that is constant across cases with different parameters cannot
 969 have that origin: it indicates that two implementations are being asked dif-
 970 ferent questions, and that the comparison rather than the implementation is
 971 at fault.

972 *How far the rule actually reaches.* Because that rule was inferred from the
 973 three incidents that produced it, it was tested against faults of known type:
 974 four convention faults and three arithmetic ones injected into the reference
 975 and scored by the study’s own metric over 15 systems of differing size and
 976 mass. The result qualifies the rule in one direction and confirms it in the
 977 other.

Table 16: Injected faults of known type, scored by relative error over 15 systems.
 Spread is the coefficient of variation across systems.

Injected fault	Kind	Mean error	Spread	Clean 1 or 2?
sign flip	convention	2.0000	0	yes
term absent	convention	1.0000	0	yes
coordinates permuted	convention	73.15	1.06	no
relative vs. absolute	convention	14.91	0.82	no
mass entry $+10^{-6}$	arithmetic	10^{-5}	0.87	no
Coriolis dropped	arithmetic	0.634	0.98	no
gravity 1% wrong	arithmetic	0.0133	0.30	no

978 **No arithmetic fault produced a clean constant (0 of 3), so the rule**

979 **does not raise false alarms.** But only two of four convention faults did.
980 Permuted coordinates and relative-versus-absolute angles both vary with con-
981 figuration exactly as an arithmetic fault would — and the second of those is
982 the third incident in the table above, which is why it was caught by inspecting
983 the mass matrix rather than by the signature.

984 **The rule is therefore sufficient but not necessary.** A clean constant
985 error is strong evidence of a convention mismatch; a varying error is no
986 evidence against one. Stated the other way round it would have dismissed
987 half the convention faults in this study as genuine numerical difference.

988 Under a relative-error metric the two commonest degenerate cases pro-
989 duce characteristic values. A term absent entirely from one side yields a
990 relative error of exactly 1; a term present with the wrong sign yields exactly
991 2, since $|a_d - a_r| = 2|a_r|$ divides out to exactly 2 wherever $|a_r| > 1$. Both
992 were observed here.

993 A second tell is that the error vanishes in a degenerate configuration and
994 grows away from it. Genuine derivation errors rarely switch on at a particular
995 system size; convention mismatches characteristically do, because degenerate
996 cases make differing conventions coincide.

997 The rule is stated for use rather than for interest, and in the direction the
998 validation supports: *in a cross-implementation comparison, a discrepancy*
999 *identical to many significant figures across cases with different parameters*
1000 *indicates a convention mismatch in the comparison, not a defect in the im-*
1001 *plementation under test.* The converse does not hold, and assuming it would
1002 have dismissed half the convention faults met here. Its practical value is that
1003 it fires early, before a spurious defect is reported. Each of the three instances

1004 here was resolved by inspecting the mass matrix directly, and each would
1005 otherwise have been reported as a defect in widely used software.

1006 **10. Implications for practice**

1007 The result is negative about a method in wide use, so it is worth stating
1008 what a practitioner should do differently. Three things follow, in decreasing
1009 order of how firmly this study supports them.

1010 *Where a closed form exists, prefer it to a second implementation.* This is
1011 the study’s firmest recommendation and the cheapest to act on. A reference
1012 derived by hand, sharing no derivation code with the system under test,
1013 detects everything a second implementation detects and additionally detects
1014 what the two would agree on. Many scientific domains admit closed forms on
1015 restricted inputs — small systems, symmetric configurations, limiting cases
1016 — even where the general problem does not. Those restricted cases are worth
1017 more as oracles than a broad suite of cross-implementation comparisons.

1018 *Ask what your implementations share, not only how they differ.* Indepen-
1019 dence is usually assessed by authorship or by codebase. Neither is the rel-
1020 evant axis for the failures reported here. Every implementation in both
1021 studies shares IEEE-754 arithmetic, and in the quadrature case they share
1022 an assumption about where to sample. The useful question before trusting a
1023 differential result is: what does every candidate assume? Shared arithmetic,
1024 shared discretisation strategy, shared library beneath nominally independent
1025 front ends, and shared mathematical idealisation are all invisible to the com-
1026 parison.

1027 *Treat exact agreement as information, not reassurance.* In both studies the
1028 failing cases produced agreement that was unusually tight: bit-identical re-
1029 turns in quadrature, and identical trajectories to displayed precision at the
1030 equilibrium here. Implementations that genuinely differ internally should dif-
1031 fer slightly in their rounding. When they do not, the reason may be that they
1032 are performing the same computation rather than that they are both right
1033 — a possibility this study met directly (§8.5) and which cost one comparison
1034 its meaning.

1035 None of this argues against differential testing, which remains the best
1036 available instrument where no oracle exists and which found nothing wrong
1037 in three mature engines here. It argues against reading agreement as evidence
1038 of correctness without asking what the agreeing parties have in common.

1039 11. Conclusion

1040 Three independently developed rigid-body dynamics engines were com-
1041 pared under adjudication by a closed-form reference sharing no derivation
1042 code with any of them, across a frozen adversarial case matrix, three struc-
1043 turally distinct mechanism families, both dynamical regimes, and constrained
1044 and unconstrained pathways.

1045 They agree to the limit of double-precision arithmetic. No engine returned
1046 an incorrect answer while reporting success on any adjudicated case, and
1047 every engine that failed did so by not returning rather than by returning
1048 something wrong.

1049 An apparent difference in reach — the largest system each engine would
1050 answer — proved on examination to measure how each engine had been

1051 driven rather than what it could do. Given the best path through each
1052 engine’s own public interface and an equal budget, all three answer systems
1053 of 80 coordinates (§6.1). That correction is reported rather than quietly
1054 applied, because a study of measurements that mislead has no standing to
1055 conceal one of its own.

1056 The single failure observed is shared by all three engines and by the
1057 independent reference. At an unstable equilibrium every implementation
1058 reports success while returning a trajectory the exact solution does not have,
1059 and none warns. Its cause lies in the interaction between the problem and
1060 finite-precision arithmetic, not in any implementation.

1061 The consequence for method is the paper’s main contribution. Differen-
1062 tial testing between implementations is a strong instrument for the failures
1063 implementations make independently, and this study found none of those
1064 in three mature engines. It is structurally blind to the failures they share.
1065 Where the shared failure arises from arithmetic rather than from code, adding
1066 implementations does not help, because the reference exhibits the failure too.
1067 Detecting such cases requires reasoning about the problem — here, recog-
1068 nising that the initial condition is an unstable equilibrium — rather than
1069 comparing implementations of it.

1070 That the engines agree is a reassuring result for practitioners who rely on
1071 them. That they agree even when all of them are wrong is the more useful
1072 one.

Disclosure

The author maintains one of the three engines measured. This is disclosed in every document of the study. The mitigation is structural: no engine adjudicates itself or any other, and the referee shares no derivation code with any of them. The case matrix was frozen before measurement under a rule forbidding post hoc removal of cases, and §8.5 reports the measurements on which the author’s engine performs worst.

Use of AI assistance. An AI assistant (Anthropic Claude) was used for portions of the implementation, the analysis, and the drafting of parts of this manuscript. The study design, the adjudication criteria, the freeze protocol, the interpretation of the results, and every decision about what is and is not claimed are the author’s, as is responsibility for the whole. Every reported figure was produced by executing committed code and checked against the run record.

Post-freeze changes to the author’s engine. Diagnosing the reach confound located two defects in the author’s engine: a dominant cost in a cosmetic simplification step, and a configuration control that reported bounding that cost while having no effect. Both were corrected after measurement. **The corrections are not present in the version measured here**, which remains pinned for the study; no figure in this paper reflects them. They are reported separately so that the improvement to the author’s own tool is not presented as a result of a study in which that tool is a subject.

Artefacts. Every figure reported was produced by executing committed code. Artefacts — the case matrix, the reference implementations, the engine

1097 adapters, the sweep scripts, and the raw output records — are available
1098 at [doi:10.5281/zenodo.22315172](https://doi.org/10.5281/zenodo.22315172), archived under the MIT licence. The
1099 repository is at <https://github.com/MechanicsDSL/mechanicsdsl> (study
1100 material under `stress_suite/`).

1101 **Funding**

1102 This research did not receive any specific grant from funding agencies in
1103 the public, commercial, or not-for-profit sectors.

1104 **CRedit authorship contribution statement**

1105 **Noah Parsons:** Conceptualization, Methodology, Software, Validation,
1106 Formal analysis, Investigation, Data curation, Writing — original draft, Writ-
1107 ing — review and editing, Visualization.

1108 **Declaration of competing interest**

1109 The author declares the following competing interest. **The author is**
1110 **the maintainer and principal developer of MechanicsDSL, one of**
1111 **the three engines evaluated in this study.** The author has no financial
1112 interest in MechanicsDSL, which is distributed without charge under the
1113 MIT licence, and no financial or personal relationship with the developers or
1114 distributors of the other two engines evaluated.

1115 The interest is mitigated structurally rather than by declaration alone,
1116 as set out in §3.7: no engine adjudicates itself or any other, the reference
1117 shares no derivation code with any engine under test, the case matrix was
1118 frozen before measurement under a rule forbidding removal of cases after

1119 their results were seen, and the reported results include every measurement
1120 on which the author’s engine performs worst (§8.5).

1121 **Data availability**

1122 All data and code required to reproduce every figure and table in this arti-
1123 cle are publicly archived under the MIT licence [7] at [doi:10.5281/zenodo.](https://doi.org/10.5281/zenodo.22315172)
1124 [22315172](https://doi.org/10.5281/zenodo.22315172). The archive contains the frozen case matrix, the closed-form
1125 reference implementations, the engine adapters, the sweep and validation
1126 scripts, and the raw output records from which every reported figure was com-
1127 puted. The source repository is at [https://github.com/MechanicsDSL/](https://github.com/MechanicsDSL/mechanicsdsl)
1128 [mechanicsdsl](https://github.com/MechanicsDSL/mechanicsdsl), with the study material under `stress_suite/`. **Appendix A**
1129 **maps each reported result to the script that produces it and the**
1130 **record it is stored in.**

1131 **Declaration of generative AI and AI-assisted technologies in the** 1132 **writing process**

1133 During the preparation of this work the author used Anthropic Claude
1134 in order to draft and edit portions of the manuscript text, after which the
1135 author reviewed and edited the content as needed and takes full responsibility
1136 for the content of the publication.

1137 Use of the same tool in the research itself — in implementing the test
1138 harness and reference implementations, and in analysing the resulting mea-
1139 surements — is described as part of the methodology in §3.7 and in the
1140 Disclosure statement above, and is not covered by this declaration.

1141 Appendix A. Reproduction map

1142 Every quantitative claim in this paper is produced by a script in the
 1143 archived artefact and recorded in a file that the archive also contains. Ta-
 1144 ble A.1 gives the mapping; its caption states the path and environment con-
 1145 ventions.

1146 Two entries record `stdout` rather than a file: those scripts print their
 1147 verdicts and were not written to emit a machine-readable record. The figures
 1148 are generated from the JSON records rather than from transcribed values,
 1149 so a figure cannot drift from the table it illustrates.

Table A.1: Reproduction map. Scripts live in `stress_suite/` and are run with `PYTHONPATH=../src`; the `.py` suffix is omitted. Records live in `stress_suite/out/` unless a path is given. † marks scripts requiring the WSL environment, since Drake has no native Windows build. Archive: [doi:10.5281/zenodo.22315172](https://doi.org/10.5281/zenodo.22315172).

Result	Script	Record
Frozen baseline, 44/0/1/10 over 55 cases (§3.3)	<code>run</code>	<code>results.json</code> , <code>report.md</code>
Equations of motion, Table 3	<code>compare_engines</code> †	<code>stdout</code>
Probe robustness, Table 4, Fig. 3	<code>probe_robustness</code> †	<code>probe_robustness.json</code>
Amplitude axis, Table 5	<code>sweep_amplitude</code>	<code>amplitude.json</code>
Chaotic regime, Table 6	<code>compare_trajectories</code> <code>-amp 3.0</code> †	<code>traj_chaotic.json</code>
Constrained pathway, Table 7	<code>check_constrained</code>	<code>stdout</code>
Cart-pole family, Table 9	<code>sweep_families</code> †	<code>families.json</code>
Slider-crank family, Table 10	<code>sweep_slidercrank</code> †	<code>slidercrank.json</code>
Frozen scaling ladder, Table 11	<code>sweep_scaling -wall</code> <code>120</code> †	<code>scaling.json</code>

continued on the next page

Result	Script	Record
Equal-budget reach, Table 12, Figs. 2 and 4	sweep_equal_budget -ladder 40,60,80†	equal_budget.json
Precision scaling, Table 13, Fig. 1	precision_equilibrium	precision_ equilibrium.json
Shared failure, cart-pole (§7.4)	shared_failure_second_ family†	shared_failure_ cartpole.json
Referee vs. hanging-chain spectrum (§3.2)	validate_reference_ physics	reference_physics_ validation.json
Accuracy across families (§6.4)	accuracy_across_ families†	accuracy_across_ families.json
Hamiltonian pathway coverage (§5.6)	hamiltonian_coverage	hamiltonian_ coverage.json
Diagnostic validation, Table 16	validate_diagnostic	diagnostic_ validation.json
Referee self-tests (11 checks)	reference	stdout
All four figures	make_figures	docs/figures/ fig1-fig4.pdf

1150 The frozen case matrix is reproduced by `run.py` alone; its verdicts are
1151 bit-identical across independent runs on the pinned engine commit, which is
1152 the gate the freeze protocol of §3.3 requires.

1153 References

- 1154 [1] American Society of Mechanical Engineers. *ASME V&V 10-2019: Stan-*
1155 *dard for Verification and Validation in Computational Solid Mechanics.*
1156 ASME, New York, 2019.
- 1157 [2] J. Chen, J. Patra, M. Pradel, Y. Xiong, H. Zhang, D. Hao, and L. Zhang.
1158 A survey of compiler testing. *ACM Computing Surveys*, 53(1):1–36, 2020.
1159 [doi:10.1145/3363562](https://doi.org/10.1145/3363562)

- 1160 [3] V. Le, M. Afshari, and Z. Su. Compiler validation via equivalence mod-
 1161 ulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on*
 1162 *Programming Language Design and Implementation (PLDI)*, pages 216–
 1163 226, 2014. [doi:10.1145/2594291.2594334](https://doi.org/10.1145/2594291.2594334)
- 1164 [4] N. Parsons. A shared blind spot in numerical quadrature. Zenodo, v1.0,
 1165 September 2026. [doi:10.5281/zenodo.22557771](https://doi.org/10.5281/zenodo.22557771)
- 1166 [5] H. V. Pham, T. Lutellier, W. Qi, and L. Tan. CRADLE: Cross-
 1167 backend validation to detect and localize bugs in deep learning li-
 1168 braries. In *Proceedings of the 41st IEEE/ACM International Con-*
 1169 *ference on Software Engineering (ICSE)*, pages 1027–1038, 2019.
 1170 [doi:10.1109/ICSE.2019.00107](https://doi.org/10.1109/ICSE.2019.00107)
- 1171 [6] X. Yang, Y. Chen, E. Eide, and J. Regehr. Finding and understanding
 1172 bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Con-*
 1173 *ference on Programming Language Design and Implementation (PLDI)*,
 1174 pages 283–294, 2011. [doi:10.1145/1993498.1993532](https://doi.org/10.1145/1993498.1993532)
- 1175 [7] N. Parsons. Silent-failure study: case matrix, reference imple-
 1176 mentations, adapters and run records [dataset]. Zenodo, 2026.
 1177 [doi:10.5281/zenodo.22315172](https://doi.org/10.5281/zenodo.22315172)
- 1178 [8] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The oracle
 1179 problem in software testing: A survey. *IEEE Transactions on Software*
 1180 *Engineering*, 41(5):507–525, 2015. [doi:10.1109/TSE.2014.2372785](https://doi.org/10.1109/TSE.2014.2372785)
- 1181 [9] T. Erez, Y. Tassa, and E. Todorov. Simulation tools for model-based
 1182 robotics: Comparison of Bullet, Havok, MuJoCo, ODE and PhysX. In

- 1183 *IEEE International Conference on Robotics and Automation (ICRA)*,
1184 pages 4397–4404, 2015.
- 1185 [10] F. González, D. Dopico, R. Pastorino, et al. Behaviour of augmented
1186 Lagrangian and Hamiltonian methods for multibody dynamics in the
1187 proximity of singular configurations. *Nonlinear Dynamics*, 85:1491–1508,
1188 2016. doi:10.1007/s11071-016-2774-5
- 1189 [11] L. Hatton and A. Roberts. How accurate is scientific software? *IEEE*
1190 *Transactions on Software Engineering*, 20(10):785–797, 1994.
- 1191 [12] IFToMM Technical Committee for Multibody Dynamics. Library of
1192 computational benchmark problems. [https://www.iftomm-multibody.](https://www.iftomm-multibody.org/benchmark)
1193 [org/benchmark](https://www.iftomm-multibody.org/benchmark). Accessed September 2026.
- 1194 [13] U. Kanewala and J. M. Bieman. Testing scientific software: A systematic
1195 literature review. *Information and Software Technology*, 56(10):1219–
1196 1232, 2014. doi:10.1016/j.infsof.2014.05.006
- 1197 [14] J. C. Knight and N. G. Leveson. An experimental evaluation of the
1198 assumption of independence in multiversion programming. *IEEE Trans-*
1199 *actions on Software Engineering*, SE-12(1):96–109, 1986.
- 1200 [15] W. M. McKeeman. Differential testing for software. *Digital Technical*
1201 *Journal*, 10(1):100–107, 1998.
- 1202 [16] W. L. Oberkampf and T. G. Trucano. Verification and validation in com-
1203 putational fluid dynamics. *Progress in Aerospace Sciences*, 38(3):209–
1204 272, 2002. doi:10.1016/S0376-0421(02)00005-2

- 1205 [17] W. L. Oberkampf and C. J. Roy. *Verification and Validation in Scientific*
1206 *Computing*. Cambridge University Press, Cambridge, 2010. ISBN 978-
1207 0-521-11360-1.
- 1208 [18] P. J. Roache. *Verification and Validation in Computational Science and*
1209 *Engineering*. Hermosa Publishers, Albuquerque, NM, 1998. ISBN 978-
1210 0-913478-08-0.
- 1211 [19] P. J. Roache. Code verification by the method of manufactured solutions.
1212 *Journal of Fluids Engineering*, 124(1):4–10, 2002. [doi:10.1115/1.1436090](https://doi.org/10.1115/1.1436090)